EXCEPTION HANDLING

1	Gracefully handle missing file & always close	try-with-resources, catch IOException	L1	Junior Backend, Java Developer
2	Prevent division by zero in calculator	Guard clause; ArithmeticException	L1	Junior Backend, Full Stack
3	Wallet “insufficient funds”	Custom InsufficientFundsException	L2	Backend Core, Fintech Developer
4	Validate scores 0–100	Custom ScoreOutOfRangeException	L1	Backend Core, EdTech
5	Book unavailability in rental	BookUnavailableException	L1	Backend Core
6	Reject temperature below −273.15°C	InvalidTemperatureException	L1	Backend Core, IoT Developer
7	Prevent zero/negative quantity in cart	IllegalArgumentException	L1	Backend Core, E-commerce
8	Detect overspeed violations	OverspeedViolationException	L1	Backend Core, IoT Developer
9	Domain-specific division-by-zero	DivisionByZeroException	L1	Backend Core
10	Order exceeds warehouse stock	OutOfStockException	L1	Backend Core, Supply Chain
11	Borrow beyond limit	BorrowLimitExceededException	L1	Backend Core
12	Withdrawal exceeds balance	InsufficientFundsException	L1	Fintech Developer
13	Booking after event start	LateBookingException	L1	Backend Core, Event App
14	Delivery beyond radius	OutOfServiceAreaException	L1	Backend Core, Logistics
15	Enroll in full class	CapacityFullException	L1	Backend Core, EdTech

16	Auction bid not higher	InvalidBidException	L1	Backend Core, Marketplace
17	Train already departed	TrainDepartedException	L1	Backend Core, Travel App
18	Warehouse order > inventory	OutOfStockException	L1	Backend Core
19	Assign ride to busy driver	DriverBusyException	L1	Backend Core, Ride-hailing
20	Late exam submission	LateSubmissionException	L1	Backend Core, EdTech
21	Overbooked event seats	CapacityFullException	L1	Backend Core, Ticketing
22	Gym access expired	MembershipExpiredException	L1	Backend Core, Fitness Apps
23	File upload too large	FileTooLargeException	L1	Backend Core
24	Hotel booking for past date	InvalidDateException	L1	Backend Core, Travel Apps
25	Max courses per student	RegistrationLimitExceededException	L1	Backend Core, EdTech
26	Late exam submission (time compare)	LateSubmissionException	L1	Backend Core
27	Order when stock depleted	OutOfStockException	L1	Backend Core, Retail
28	Enforce credit limit	CreditLimitExceededException	L2	Backend Core, Fintech
29	Game room full	RoomFullException	L1	Backend Core, Gaming
30	Lock account after failed logins	AccountLockedException	L2	Backend Core, Security Engineer

31	Double-book airline seat	SeatAlreadyBookedException	L2	Backend Core, Travel Apps
32	Block payments below minimum	MinimumAmountException	L1	Backend Core, Fintech
33	Prevent re-enrollment of completed course	AlreadyCompletedException	L1	Backend Core, EdTech
34	Remove more inventory than available	InventoryUnderflowException	L1	Backend Core, Retail
35	Underage car rental driver	UnderageDriverException	L1	Backend Core, Mobility Apps
36	Region-restricted content	RegionRestrictedException	L2	Backend Core, Media Apps
37	Duplicate job application	DuplicateApplicationException	L1	Backend Core, HRTech
38	Resilient multi-thread document processing	UncaughtExceptionHandler	L3	Backend Core, Concurrency
39	Retry flaky payment up to 3 times	Retry with backoff; PaymentFailedException	L2	Backend Core, Fintech
40	Block bids below min amount	MinimumBidException	L1	Backend Core, Marketplace
41	Reject order qty > stock	OutOfStockException	L1	Backend Core, Supply Chain
42	Handle invalid signups & continue	Per-item validation; ValidationException	L2	Backend Core, SaaS Apps
43	Retry connecting to remote server	Retry/backoff; IOException	L2	Backend Core, Cloud Systems

44	Validate profile updates; report missing fields	ValidationException	L2	Backend Core, User Services
45	Process multiple file uploads, report violations	Per-file try/catch	L2	Backend Core
46	Batch processing: skip corrupted records	Per-record try/catch; log & continue	L2	Backend Core, Data Processing
47	Image pipeline resilience	Try/catch per filter; isolate errors	L2	Backend Core, Media Processing
48	Failed payments shouldn’t block queue	Catch in worker threads	L2	Backend Core, Fintech
49	Partial book request success	Aggregate exceptions	L2	Backend Core
50	Scheduler — failing task isolation	ScheduledExecutorService, catch inside	L2	Backend Core, Scheduler Systems
51	Partial sensor data collection	Fan-in results; per-sensor try/catch	L2	Backend Core, IoT Systems

Scenario 1: Handling missing file during read

In real-world applications like payroll processing or bank statement imports, files might not always exist in the expected directory. If the program crashes abruptly without a clear message, end-users and support teams get frustrated. The goal here is to handle the missing file gracefully and make sure system resources are always cleaned up.

Answer:

The safest way is to use Java's try-with-resources (introduced in Java 7). It ensures that the file resource is closed automatically once the block is finished, even if an exception occurs in between. If we don't use try-with-resources, then a finally block is required to close the resource manually.

Code:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileNotFoundException;
import java.io.IOException;

public class ReadFileApp {
    public static void main(String[] args) {
        // Example: trying to load records.txt
        try (BufferedReader reader = new BufferedReader(new
FileReader("records.txt"))) {
            String firstLine = reader.readLine();
            System.out.println("Loaded: " + firstLine);
        } catch (FileNotFoundException e) {
            // Specific handling for file missing
            System.out.println("File not found. Please check if 'records.txt' exists in the
folder.");
        } catch (IOException e) {
            // Covers read errors, corrupted files, etc.
            System.out.println("Error while reading the file: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- I used try-with-resources so I don't have to explicitly close the BufferedReader. It avoids memory leaks and resource exhaustion.

- I separated FileNotFoundException from IOException to give a clear and user-friendly error message for missing files, instead of a generic “read error”.
- In real-world systems (say a ticket booking system loading price data), this prevents confusion when files are missing due to misconfiguration or deployment issues.

Future scope:

- Add logging frameworks like SLF4J/Logback instead of System.out.println so errors can be monitored in production.
- Implement a retry mechanism or fallback (e.g., load a default file or fetch data from DB).
- Externalize file paths into configuration files, rather than hardcoding "records.txt".
- If large files are processed, use Files.lines(Path) with Streams for better scalability.

Scenario 2: Preventing division by zero in a calculator

In a simple calculator program, dividing by zero is a very common input error. Instead of letting the program crash with an ArithmeticException, the system should guide the user to re-enter a valid divisor.

Answer:

I'd put the division inside a loop, use a try-catch for error handling, and keep asking the user until they enter a valid non-zero divisor. This ensures smooth user experience without abrupt termination.

Code:

```
import java.util.Scanner;

public class SafeDivision {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        while (true) {
            try {
                System.out.print("Enter the dividend: ");
                int a = scan.nextInt();

                System.out.print("Enter the divisor: ");
                int b = scan.nextInt();

                int result = a / b; // risky line
                System.out.println("Quotient: " + result);

                break; // exit loop once successful
            } catch (ArithmeticException e) {
                System.out.println("Error: divisor cannot be zero. Please enter again.");
            }
        }

        scan.close();
    }
}
```

Reasoning to interviewer:

- The risky operation is a / b, so I wrapped only that part with exception handling.
- Instead of letting the program crash, I inform the user what went wrong in a human-readable way.
- A loop ensures the program keeps running until correct input is given, which is critical for interactive apps like ATMs or calculators.

Future scope:

- Validate non-numeric input using InputMismatchException.
- Extract input logic into a method for cleaner design.
- Add support for floating-point division for real-world use.
- Extend to other operations (add, subtract, multiply) and build a full-featured calculator.

Scenario 3: Handling insufficient balance in a digital wallet

In a wallet application, users might try withdrawing more money than their current balance. Instead of using a generic exception, we should throw a meaningful, domain-specific exception that clearly conveys the business rule violation.

Answer:

I'd define a custom exception class called LowFundsException. Whenever a withdrawal request exceeds available balance, I'll throw this exception with a descriptive message. The calling code can then catch it and show a proper user-friendly message.

Code:

```
class LowFundsException extends Exception {
    public LowFundsException(String message) {
        super(message);
    }
}

public class Wallet {
    private double funds;

    public Wallet(double initialFunds) {
        this.funds = initialFunds;
    }

    public void withdraw(double amount) throws LowFundsException {
        if (amount > funds) {
            throw new LowFundsException(
                "Transaction declined: Available balance = " + funds + ", Requested = "
+ amount
            );
        }
        funds -= amount;
        System.out.println("Withdrawal successful. Remaining balance: " + funds);
    }

    public static void main(String[] args) {
        Wallet myWallet = new Wallet(100);

        try {
            myWallet.withdraw(150);
        } catch (LowFundsException e) {
            System.out.println(e.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- I created a checked exception by extending Exception because insufficient funds is a recoverable situation (the program should continue, just not process the transaction).
- The exception message is descriptive, showing both available and requested amounts, which helps users understand the issue instantly.
- This design separates business rule violations (low funds) from technical issues (I/O, nulls), making the codebase cleaner.

Future scope:

- Extend the wallet with deposit functionality.
- Add transaction history and timestamp logging for auditing.
- Convert the exception to RuntimeException if we want less boilerplate in higher layers.
- Integrate with services like SMS/email to notify users of failed withdrawals.

Scenario 4: Validating student scores in a grading portal

In a grading system, it's common for users (or even external systems) to enter invalid scores, like -5 or 120. Instead of letting such values silently pass, the system should enforce clear validation rules. Using a custom exception makes the intent explicit and keeps the error reporting meaningful.

Answer:

I'd create a custom exception class called OutOfRangeScoreException. Whenever a score is outside 0–100, I'll throw this exception with a descriptive message. This way, only valid scores are processed and invalid cases are clearly separated from technical errors.

Code:

```
class OutOfRangeScoreException extends Exception {
    public OutOfRangeScoreException(String message) {
        super(message);
    }
}

public class GradeValidator {
    public void checkScore(int score) throws OutOfRangeScoreException {
        if (score < 0 || score > 100) {
            throw new OutOfRangeScoreException(
                "Score " + score + " is invalid. Allowed range: 0–100."
            );
        }
        System.out.println("Score accepted: " + score);
    }

    public static void main(String[] args) {
        GradeValidator validator = new GradeValidator();

        try {
            validator.checkScore(120); // invalid case
        } catch (OutOfRangeScoreException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- A custom checked exception highlights that invalid input is a recoverable business error, not a system crash.
- Including the actual score and valid range in the message gives immediate clarity to the user or support team.
- This separation of concerns makes the system robust—invalid data never enters the grading process.

Future scope:

- Add batch validation for multiple scores (e.g., from CSV uploads).
- Support international grading systems with configurable min–max ranges.
- Integrate with UI forms to highlight invalid fields in real-time.

Scenario 5: Handling unavailable books in a rental system

In a book rental application, two users might try to borrow the same book. Instead of using a generic exception, we should signal this situation with a domain-specific one that clearly conveys book unavailability.

Answer:

I'd create a custom exception BookUnavailableException. Whenever a borrow request comes in for an already loaned-out title, this exception is raised. That makes the code self-explanatory and easier to maintain.

Code:

```
class BookUnavailableException extends Exception {
    public BookUnavailableException(String message) {
        super(message);
    }
}

class BookLending {
    private boolean available = false; // default state: book is not available

    public void borrow() throws BookUnavailableException {
        if (!available) {
            throw new BookUnavailableException("Sorry, this book is currently
checked out.");
        }
        System.out.println("You have borrowed the book successfully.");
        available = false; // mark as unavailable after borrowing
    }

    public void returnBook() {
        available = true;
        System.out.println("Book returned and available for others now.");
    }

    public static void main(String[] args) {
        BookLending lending = new BookLending();

        try {
            lending.borrow(); // will throw exception
        } catch (BookUnavailableException e) {
            System.out.println(e.getMessage());
        }

        lending.returnBook(); // make it available again
    }
```

```
}
```

Reasoning to interviewer:

- The custom exception makes it explicit that the error relates to book availability, not a generic runtime issue.
- Borrow and return operations update availability consistently, preventing duplicate checkouts.
- This design reflects real-world business rules, where availability is a stateful property of a resource.

Future scope:

- Extend the model to handle multiple copies of the same title.
- Track due dates and penalties by enriching the exception message.
- Integrate with a reservation queue so users can waitlist books instead of failing outright.

Scenario 6: Validating temperatures in a weather monitoring program

In a weather station, we often deal with Celsius and Fahrenheit readings. However, temperatures below absolute zero are physically impossible. If such a value enters the system, it indicates either a sensor fault or bad input. Instead of silently accepting it, the system should throw a meaningful exception.

Answer:

I'd define a custom exception called InvalidTemperatureException. Whenever a value is below absolute zero (-273.15°C), the exception is raised. This ensures that invalid readings don't pollute the system and operators get a clear message about what went wrong.

Code:

```
class InvalidTemperatureException extends Exception {
    public InvalidTemperatureException(String message) {
        super(message);
    }
}

public class WeatherStation {
    public static void checkTemperature(double tempCelsius) throws
InvalidTemperatureException {
        if (tempCelsius < -273.15) {
            throw new InvalidTemperatureException(
                "Invalid reading: " + tempCelsius + "°C. Must not be below -273.15°C
(absolute zero)."
            );
        }
        System.out.println("Temperature recorded: " + tempCelsius + "°C");
    }

    public static double toFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    public static double toCelsius(double fahrenheit) {
        return (fahrenheit - 32) * 5 / 9;
    }

    public static void main(String[] args) {
        try {
            checkTemperature(-290);
        } catch (InvalidTemperatureException e) {
            System.out.println(e.getMessage());
```

```
        }

        System.out.println("30°C = " + toFahrenheit(30) + "°F");
        System.out.println("86°F = " + toCelsius(86) + "°C");
    }
}
```

Reasoning to interviewer:

- The custom checked exception communicates a clear domain rule violation (temperature too low).
- I added conversion methods for Celsius ↔ Fahrenheit since monitoring programs usually support both scales.
- Validations ensure sensor errors are caught early instead of corrupting historical records or triggering false alarms.

Future scope:

- Support Kelvin as an additional unit.
- Add logging for rejected values so operators can investigate faulty sensors.
- Integrate thresholds and alerts (e.g., extreme heat or cold warnings).

Scenario 7: Preventing invalid quantity in an online cart

In e-commerce platforms, adding zero or negative quantities makes no business sense. Instead of silently failing, the system should raise a clear exception so the UI or service layer can inform the buyer immediately.

Answer:

I'd define a custom exception called QuantityNotAllowedException. If a user tries to add zero or negative items, this exception is raised. That makes the rule explicit and avoids polluting cart data with invalid entries.

Code:

```
class QuantityNotAllowedException extends Exception {
    public QuantityNotAllowedException(String message) {
        super(message);
    }
}

class OnlineCart {
    public void addItem(int qty) throws QuantityNotAllowedException {
        if (qty <= 0) {
            throw new QuantityNotAllowedException(
                "Invalid quantity: " + qty + ". Must be at least 1."
            );
        }
        System.out.println(qty + " item(s) added to your cart.");
    }

    public static void main(String[] args) {
        OnlineCart cart = new OnlineCart();

        try {
            cart.addItem(0); // invalid case
        } catch (QuantityNotAllowedException e) {
            System.out.println(e.getMessage());
        }

        try {
            cart.addItem(3); // valid case
        } catch (QuantityNotAllowedException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- This business rule is simple: quantity must be ≥ 1. By making it a custom exception, I clearly communicate the domain-specific rule.
- This ensures data integrity in the system—no orders with zero or negative items.
- In real systems, such checks often sit at the service layer before persisting to a database.

Future scope:

- Extend to validate stock limits (e.g., can’t add more than available inventory).
- Add item-level identifiers (e.g., addItem(String productId, int qty)).
- Support bulk operations like adding multiple products at once.
- Integrate with pricing logic to show updated totals immediately.

Scenario 8: Detecting overspeeding in a traffic monitoring system

In road traffic systems, speed detection is crucial. If a car goes above the legal limit, the system should not just print values—it should flag the event, so it can trigger alerts like challans or camera captures.

Answer:

I'd define a custom checked exception called OverSpeedException. Whenever a vehicle crosses the maximum allowed limit, I'll throw this exception. That way, the rule is enforced at the domain level and can later integrate with alerting mechanisms.

Code:

```
class OverSpeedException extends Exception {
    public OverSpeedException(String message) {
        super(message);
    }
}

public class TrafficMonitor {
    private int speed;
    private static final int LEGAL_LIMIT = 100; // km/h

    public void updateSpeed(int newSpeed) throws OverSpeedException {
        if (newSpeed > LEGAL_LIMIT) {
            throw new OverSpeedException(
                "Speed " + newSpeed + " km/h exceeded the legal limit of " +
LEGAL_LIMIT + " km/h."
            );
        }
        this.speed = newSpeed;
        System.out.println("Current speed: " + speed + " km/h");
    }

    public static void main(String[] args) {
        TrafficMonitor monitor = new TrafficMonitor();

        try {
            monitor.updateSpeed(115); // overspeed case
        } catch (OverSpeedException e) {
            System.out.println(e.getMessage());
        }

        try {
            monitor.updateSpeed(80); // valid case
        } catch (OverSpeedException e) {
            System.out.println(e.getMessage());
```

```
    }
  }
}
```

Reasoning to interviewer:

- Overspeeding is a **business rule violation**, not a system error, hence a custom exception fits better than a generic one.
- Throwing an exception forces callers to deal with the event, which is critical in real systems where alerts must be logged or fines issued.
- Keeping the speed field private ensures that only validated values get recorded.

Future scope:

- Integrate with logging systems to store speed violations for legal purposes.
- Add multiple vehicle tracking with IDs.
- Trigger asynchronous notifications (SMS/email) when a violation occurs.
- Make the speed limit configurable per road/zone instead of hardcoding.

Scenario 9: Preventing division by zero with a custom exception in a calculator

Division by zero is undefined mathematically, and in calculator software, allowing it would lead to crashes or misleading outputs. Instead of using ArithmeticException, we can make it clearer with a domain-specific exception.

Answer:

I'd create a custom exception called ZeroDivisionException. Whenever the denominator is zero, this exception will be raised with a descriptive message, and the app can then display a proper alert instead of crashing.

Code:

```
class ZeroDivisionException extends Exception {
    public ZeroDivisionException(String message) {
        super(message);
    }
}

public class MathTool {
    public double divideNumbers(int a, int b) throws ZeroDivisionException {
        if (b == 0) {
            throw new ZeroDivisionException("Division by zero is not permitted.");
        }
        return (double) a / b;
    }

    public static void main(String[] args) {
        MathTool tool = new MathTool();

        try {
            System.out.println(tool.divideNumbers(8, 0)); // invalid case
        } catch (ZeroDivisionException e) {
            System.out.println(e.getMessage());
        }

        try {
            System.out.println(tool.divideNumbers(8, 2)); // valid case
        } catch (ZeroDivisionException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- Using ZeroDivisionException instead of relying only on ArithmeticException makes the error message **domain-relevant** and clear to end users.
- Since calculators are user-facing applications, descriptive errors improve user experience and prevent confusion.
- By returning a double, I ensure results include fractional precision, which is important for real calculations.

Future scope:

- Extend the calculator with other operations (addition, multiplication, square roots).
- Handle invalid input types (like characters instead of numbers).
- Build a simple GUI or REST API wrapper on top of this logic.
- Log invalid attempts for analytics (how often users try dividing by zero).

Scenario 10: Preventing orders beyond available stock in a warehouse

In warehouse systems, one of the most critical checks is ensuring orders don't exceed the available product count. If such requests are allowed, the system may promise inventory that doesn't exist, leading to delivery failures and customer dissatisfaction.

Answer:

I'd create a custom exception called StockUnavailableException. If a requested order quantity is greater than the available stock, I'll throw this exception. That way, only valid orders are processed, and the system provides immediate feedback on inventory shortages.

Code:

```
class StockUnavailableException extends Exception {
    public StockUnavailableException(String message) {
        super(message);
    }
}

class Warehouse {
    private int availableStock;

    public Warehouse(int stock) {
        this.availableStock = stock;
    }

    public void processOrder(int qty) throws StockUnavailableException {
        if (qty > availableStock) {
            throw new StockUnavailableException(
                "Requested: " + qty + ", Available: " + availableStock
            );
        }
        availableStock -= qty;
        System.out.println("Order processed. Remaining stock: " + availableStock);
    }

    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse(6);

        try {
            warehouse.processOrder(9); // invalid case
        } catch (StockUnavailableException e) {
            System.out.println(e.getMessage());
        }

        try {
```

```
            warehouse.processOrder(4); // valid case
        } catch (StockUnavailableException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- A custom exception communicates that the error is business-related (lack of stock), not a technical fault.
- The available stock is updated only after successful validation, ensuring consistency.
- This approach mirrors real-world warehouse workflows where stock checks are mandatory before order fulfillment.

Future scope:

- Integrate with supplier systems to auto-reorder when stock runs low.
- Support partial fulfillment (e.g., order 10 but deliver 6 if only 6 are available).
- Maintain audit logs of failed orders for reporting and analysis.

Scenario 11: Restricting users from borrowing more than allowed in a library system

Library systems often limit how many books a user can borrow to ensure fair access for all members. If a user exceeds this limit, the system should flag it clearly.

Answer:

I'd define a custom exception MaxBooksLimitExceededException. When the user tries to borrow more books than the allowed maximum, I'd throw this exception, making it explicit that the failure is due to a rule violation, not a technical crash.

Code:

```
class MaxBooksLimitExceededException extends Exception {
    public MaxBooksLimitExceededException(String message) {
        super(message);
    }
}

class LibraryAccount {
    private int borrowedBooks = 0;
    private static final int MAX_LIMIT = 4;

    public void borrowBook(int count) throws MaxBooksLimitExceededException {
        if (borrowedBooks + count > MAX_LIMIT) {
            throw new MaxBooksLimitExceededException(
                "Borrowing limit exceeded. Max allowed: " + MAX_LIMIT +
                ", Currently borrowed: " + borrowedBooks
            );
        }
        borrowedBooks += count;
        System.out.println("Books borrowed. Total now: " + borrowedBooks);
    }

    public static void main(String[] args) {
        LibraryAccount account = new LibraryAccount();

        try {
            account.borrowBook(5); // invalid
        } catch (MaxBooksLimitExceededException e) {
            System.out.println(e.getMessage());
        }

        try {
            account.borrowBook(2); // valid
        } catch (MaxBooksLimitExceededException e) {
            System.out.println(e.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- The custom exception makes the rule violation explicit, instead of mixing it with generic errors.
- I included the current borrowed count in the message, which helps users understand the exact reason for failure.
- This approach is scalable—different account types (e.g., student vs. faculty) could have different limits.

Future scope:

- Add functionality to return books and update the count.
- Implement user types with different borrowing limits (students, staff, premium members).
- Add due-date tracking and penalties for overdue books.
- Connect with an inventory system to ensure requested books are physically available.

Scenario 12: Preventing withdrawals beyond balance in a banking app

In a banking system, withdrawals that exceed the account balance can't be allowed, as they would result in negative balances and financial inconsistencies. Instead of allowing the operation to proceed, we must explicitly stop it with a meaningful exception.

Answer:

I'd define a custom exception called LowBalanceException. Whenever the withdrawal amount is greater than the balance, this exception will be thrown with a descriptive message. This ensures withdrawals are processed only when sufficient funds exist.

Code:

```
class LowBalanceException extends Exception {
    public LowBalanceException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double startingBalance) {
        this.balance = startingBalance;
    }

    public void withdraw(double amount) throws LowBalanceException {
        if (amount > balance) {
            throw new LowBalanceException(
                "Insufficient funds. Available balance: $" + balance + ", Requested: $" +
amount
            );
        }
        balance -= amount;
        System.out.println("Withdrawal successful. New balance: $" + balance);
    }

    public static void main(String[] args) {
        BankAccount acc = new BankAccount(200);

        try {
            acc.withdraw(300); // invalid case
        } catch (LowBalanceException e) {
            System.out.println(e.getMessage());
        }
```

```
        try {
            acc.withdraw(150); // valid case
        } catch (LowBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- A custom exception like LowBalanceException makes the rule violation explicit, instead of mixing it with general exceptions.
- Including both available and requested amounts in the message gives the user context immediately.
- This reflects real-world scenarios where banking systems must prevent overdrafts unless overdraft facilities are explicitly allowed.

Future scope:

- Add deposit functionality and transaction history.
- Introduce overdraft accounts where exceptions are bypassed within limits.
- Integrate alerts (SMS/email) when failed transactions occur.
- Apply concurrency control if multiple withdrawals can happen simultaneously.

Scenario 13: Stopping ticket bookings after the event starts

In event management systems, users must not be allowed to book tickets once the event has begun. If such requests are accepted, it would lead to inconsistencies and customer disputes.

Answer:

I'd create a custom exception called LateBookingException. When a user tries to book tickets at or after the event's start time, this exception will be thrown. This ensures the booking process respects time constraints.

Code:

```
import java.time.LocalTime;

class LateBookingException extends Exception {
    public LateBookingException(String message) {
        super(message);
    }
}

class TicketService {
    private LocalTime eventStartTime = LocalTime.of(18, 0); // event starts at 6:00
PM

    public void bookTicket(LocalTime bookingTime) throws LateBookingException {
        if (!bookingTime.isBefore(eventStartTime)) {
            throw new LateBookingException(
                "Booking denied. Tickets cannot be booked at or after " +
eventStartTime
            );
        }
        System.out.println("Ticket booked successfully at " + bookingTime);
    }

    public static void main(String[] args) {
        TicketService service = new TicketService();

        try {
            service.bookTicket(LocalTime.of(18, 5)); // too late
        } catch (LateBookingException e) {
            System.out.println(e.getMessage());
        }

        try {
            service.bookTicket(LocalTime.of(17, 45)); // valid booking
        } catch (LateBookingException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```

Reasoning to interviewer:

- Using LocalTime makes the solution future-proof and avoids error-prone manual comparisons.
- The custom exception LateBookingException keeps the business rule violation separate from technical failures.
- The check uses isBefore for readability, ensuring bookings are only valid before the event time.

Future scope:

- Extend to LocalDateTime for handling multiple-day events.
- Allow configurable cut-off times (e.g., stop bookings 30 minutes before start).
- Connect with a payment gateway to rollback transactions if booking fails due to timing.
- Add a waitlist system for users trying to book late.

Scenario 14: Rejecting orders beyond serviceable delivery distance

In a food delivery platform, one of the basic constraints is distance. If a customer's location is too far, the order shouldn't even be accepted, otherwise it will cause operational issues and customer dissatisfaction.

Answer:

I'd define a custom exception called DeliveryNotAvailableException. Whenever the delivery distance exceeds the maximum allowed threshold, this exception will be raised with a clear message.

Code:

```
class DeliveryNotAvailableException extends Exception {
    public DeliveryNotAvailableException(String message) {
        super(message);
    }
}

class FoodDelivery {
    private static final int MAX_DISTANCE = 15; // serviceable range in km

    public void placeOrder(int distance) throws DeliveryNotAvailableException {
        if (distance > MAX_DISTANCE) {
            throw new DeliveryNotAvailableException(
                "Delivery not available beyond " + MAX_DISTANCE + " km."
            );
        }
        System.out.println("Order placed for delivery at " + distance + " km.");
    }

    public static void main(String[] args) {
        FoodDelivery delivery = new FoodDelivery();

        try {
            delivery.placeOrder(20); // invalid case
        } catch (DeliveryNotAvailableException e) {
            System.out.println(e.getMessage());
        }

        try {
            delivery.placeOrder(10); // valid case
        } catch (DeliveryNotAvailableException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- Business logic is enforced via a custom exception, so invalid orders are stopped before processing.
- A constant MAX_DISTANCE makes the rule explicit and easy to change later (e.g., per city).
- This approach prevents customer dissatisfaction by not accepting orders the business cannot fulfill.

Future scope:

- Make distance configurable by region or restaurant.
- Integrate with Google Maps API for accurate real-time distance checks.
- Offer pickup option for customers outside the serviceable range.
- Add dynamic surge pricing based on distance instead of a hard cutoff.

Scenario 15: Preventing enrollments when a course is full

In university systems, courses have limited capacity. Once the limit is reached, new enrollments must be rejected; otherwise, it could cause chaos in scheduling and faculty workload.

Answer:

I'd define a custom exception called CourseFullException. Whenever a student tries to enroll but no seats are left, this exception will be thrown. That keeps the capacity rule explicit and prevents over-enrollment.

Code:

```
class CourseFullException extends Exception {
    public CourseFullException(String message) {
        super(message);
    }
}

class Course {
    private int seatsAvailable;

    public Course(int seats) {
        this.seatsAvailable = seats;
    }

    public void enrollStudent() throws CourseFullException {
        if (seatsAvailable <= 0) {
            throw new CourseFullException("No seats left in the course.");
        }
        seatsAvailable--;
        System.out.println("Student enrolled. Remaining seats: " + seatsAvailable);
    }

    public static void main(String[] args) {
        Course course = new Course(0); // course already full

        try {
            course.enrollStudent(); // invalid case
        } catch (CourseFullException e) {
            System.out.println(e.getMessage());
        }

        Course course2 = new Course(2); // course with seats
        try {
            course2.enrollStudent(); // valid case
            course2.enrollStudent(); // valid case
            course2.enrollStudent(); // invalid case
        } catch (CourseFullException e) {
```

```
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- Throwing a domain-specific exception keeps the error handling clear: failure is due to capacity, not a technical bug.
- The check ensures course seat counts remain consistent—every successful enrollment decrements the count.
- This reflects real-world workflows where admins must control course sizes.

Future scope:

- Add a waitlist system for students who try enrolling after the course is full.
- Notify students when seats become available due to cancellations.
- Support different maximum capacities for labs, lectures, or online classes.
- Integrate with a payment/registration system to ensure atomic seat booking.

Scenario 16: Enforcing higher bids in an online auction system

In auctions, fairness depends on each new bid being strictly higher than the current one. If lower or equal bids are accepted, it breaks the logic of competitive bidding and confuses participants.

Answer:

I'd create a custom exception BidTooLowException. Whenever a user places a bid less than or equal to the current highest bid, the exception will be raised. This way, the system enforces the rule and provides clear feedback to users.

Code:

```
class BidTooLowException extends Exception {
    public BidTooLowException(String message) {
        super(message);
    }
}

class AuctionItem {
    private double highestBid = 500.0; // starting bid

    public void placeBid(double bidAmount) throws BidTooLowException {
        if (bidAmount <= highestBid) {
            throw new BidTooLowException(
                "Bid too low. Current highest bid is $" + highestBid + ". Your bid: $" +
bidAmount
            );
        }
        highestBid = bidAmount;
        System.out.println("Bid accepted! New highest bid: $" + highestBid);
    }

    public static void main(String[] args) {
        AuctionItem item = new AuctionItem();

        try {
            item.placeBid(450.0); // invalid
        } catch (BidTooLowException e) {
            System.out.println(e.getMessage());
        }

        try {
            item.placeBid(600.0); // valid
        } catch (BidTooLowException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The exception makes it explicit that rejection is due to a rule violation, not a technical failure.
- By storing the highestBid in the item itself, I encapsulate state and ensure each auction item enforces its own rules.
- The message includes both the current and attempted bid to give the user immediate context.

Future scope:

- Track bidder IDs so the system records who placed the highest bid.
- Add bid increment rules (e.g., new bids must be at least $10 higher).
- Introduce concurrency handling so multiple users bidding at the same time are properly synchronized.
- Connect with payment services to authorize funds before accepting a bid.

Scenario 17: Preventing bookings for trains that have already departed

In train ticketing systems, users must not be allowed to book seats on trains that have already left. If this check is missing, it could result in invalid transactions and poor user experience.

Answer:

I'd define a custom exception TrainDepartedException. When a user attempts to book a ticket at or after the departure time, this exception will be raised with a descriptive message.

Code:

```
import java.time.LocalTime;

class TrainDepartedException extends Exception {
    public TrainDepartedException(String message) {
        super(message);
    }
}

class TrainBooking {
    private LocalTime departureTime = LocalTime.of(9, 30); // train leaves at 9:30
AM

    public void bookSeat(LocalTime bookingTime) throws TrainDepartedException {
        if (!bookingTime.isBefore(departureTime)) {
            throw new TrainDepartedException(
                "Booking denied. The train already departed at " + departureTime
            );
        }
        System.out.println("Booking confirmed for " + bookingTime);
    }

    public static void main(String[] args) {
        TrainBooking booking = new TrainBooking();

        try {
            booking.bookSeat(LocalTime.of(9, 35)); // too late
        } catch (TrainDepartedException e) {
            System.out.println(e.getMessage());
        }

        try {
            booking.bookSeat(LocalTime.of(9, 15)); // valid booking
        } catch (TrainDepartedException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```

Reasoning to interviewer:

- Using LocalTime keeps comparisons simple and avoids manual calculations.
- The condition !bookingTime.isBefore(departureTime) ensures bookings are blocked at or after departure.
- A descriptive message tells the user exactly why their booking failed.

Future scope:

- Switch to LocalDateTime for real-world schedules involving dates.
- Allow cut-off windows (e.g., no booking within 15 minutes of departure).
- Integrate with refund logic for failed or late booking attempts.
- Add dynamic seat allocation and train capacity checks alongside departure validation.

Scenario 18: Preventing orders that exceed available inventory in a warehouse

In warehouse management, overselling stock is a critical issue. If orders are accepted beyond actual inventory, it creates delivery failures and unhappy customers. The system should enforce inventory checks before confirming any order.

Answer:

I'd maintain a running stock count and throw a custom exception StockUnavailableException if the requested quantity is greater than what's available. This ensures inventory integrity and avoids over-promising.

Code:

```
class StockUnavailableException extends Exception {
    public StockUnavailableException(String message) {
        super(message);
    }
}

class Warehouse {
    private int stock = 30; // initial stock

    public void processOrder(int quantity) throws StockUnavailableException {
        if (quantity > stock) {
            throw new StockUnavailableException(
                "Requested quantity (" + quantity +
                ") exceeds available stock (" + stock + ")"
            );
        }
        stock -= quantity;
        System.out.println("Order processed. Remaining stock: " + stock);
    }

    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse();

        try {
            warehouse.processOrder(40); // invalid case
        } catch (StockUnavailableException e) {
            System.out.println(e.getMessage());
        }

        try {
            warehouse.processOrder(10); // valid case
        } catch (StockUnavailableException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```

Reasoning to interviewer:

- The check ensures business rules are enforced before reducing stock.
- A descriptive exception message clarifies both requested and available quantities.
- Encapsulation (stock as private) ensures updates only happen through validated logic.

Future scope:

- Support partial fulfillment (ship whatever is in stock).
- Integrate with suppliers for automatic restocking when levels drop.
- Add concurrency control if multiple orders hit the system at once.
- Maintain historical logs of stock shortages for reporting.

Scenario 19: Preventing ride assignments when a driver is unavailable

In ride-hailing systems, drivers cannot handle more than one trip at a time. If the system assigns a busy driver, both customers are impacted. A clear availability check is mandatory.

Answer:

I'd maintain a driver's status (available or engaged) and throw a custom exception DriverNotAvailableException when an attempt is made to assign a busy driver.

Code:

```
class DriverNotAvailableException extends Exception {
    public DriverNotAvailableException(String message) {
        super(message);
    }
}

class RideService {
    private boolean driverAvailable = false; // driver is currently busy

    public void assignDriver() throws DriverNotAvailableException {
        if (!driverAvailable) {
            throw new DriverNotAvailableException("Driver is currently on another
trip.");
        }
        System.out.println("Driver assigned successfully.");
        driverAvailable = false; // mark driver as busy after assignment
    }

    public void freeDriver() {
        driverAvailable = true;
        System.out.println("Driver is now available for new rides.");
    }

    public static void main(String[] args) {
        RideService service = new RideService();

        try {
            service.assignDriver(); // invalid, driver not available
        } catch (DriverNotAvailableException e) {
            System.out.println(e.getMessage());
        }

        service.freeDriver(); // mark driver as free

        try {
            service.assignDriver(); // valid case
```

```
        } catch (DriverNotAvailableException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- By tracking driver availability with a boolean, we avoid double-booking.
- The exception separates business errors (driver already engaged) from technical ones.
- Adding freeDriver() makes the lifecycle realistic: once a trip ends, the driver becomes available again.

Future scope:

- Replace boolean with an enum (AVAILABLE, ON_TRIP, OFFLINE) for richer states.
- Support assigning from a pool of drivers instead of one.
- Add concurrency handling so multiple ride requests don’t assign the same driver simultaneously.
- Integrate with GPS for smarter assignments based on proximity.

Scenario 20: Preventing submissions after the exam timer ends

In online exam platforms, strict timing rules ensure fairness. Once the exam window closes, late submissions must be rejected—even if the student tries to submit just a few seconds after the deadline.

Answer:

I'd keep track of the exam's end time. When a student submits, I'd compare their submission time with the end time. If it's after the allowed window, I'd throw a TimeLimitExceededException with a clear message.

Code:

```
import java.time.LocalTime;

class TimeLimitExceededException extends Exception {
    public TimeLimitExceededException(String message) {
        super(message);
    }
}

class OnlineExam {
    private LocalTime endTime = LocalTime.of(11, 0); // exam ends at 11:00 AM

    public void submitExam(LocalTime submissionTime) throws
TimeLimitExceededException {
        if (submissionTime.isAfter(endTime)) {
            throw new TimeLimitExceededException(
                "Submission failed. Exam ended at " + endTime
            );
        }
        System.out.println("Exam submitted successfully at " + submissionTime);
    }

    public static void main(String[] args) {
        OnlineExam exam = new OnlineExam();

        try {
            exam.submitExam(LocalTime.of(11, 15)); // late submission
        } catch (TimeLimitExceededException e) {
            System.out.println(e.getMessage());
        }

        try {
            exam.submitExam(LocalTime.of(10, 55)); // valid submission
        } catch (TimeLimitExceededException e) {
            System.out.println(e.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- The comparison is simple: submissionTime.isAfter(endTime). This avoids edge-case confusion.
- By using LocalTime, I focused on time-of-day, but in production, I'd use LocalDateTime for multi-day exams.
- The custom exception separates business rule violations from technical failures, making error handling cleaner.

Future scope:

- Add auto-submit when the timer ends to avoid manual late submissions.
- Support time extensions for special cases (like accessibility needs).
- Log late submission attempts for audit trails.
- Handle time zones properly for international students.

Scenario 21: Preventing overbooking of event seats

In ticket booking systems, overselling seats leads to customer disputes and operational chaos. The system should enforce seat limits before confirming bookings.

Answer:

I'd maintain a seat counter for each event. If a booking request exceeds the available seats, I'd throw a SeatsUnavailableException. This ensures users only get valid confirmations and the system stays consistent.

Code:

```
class SeatsUnavailableException extends Exception {
    public SeatsUnavailableException(String message) {
        super(message);
    }
}

class EventBooking {
    private int remainingSeats = 25; // initial seat count

    public void bookSeats(int seats) throws SeatsUnavailableException {
        if (seats > remainingSeats) {
            throw new SeatsUnavailableException(
                "Only " + remainingSeats + " seats available. Requested: " + seats
            );
        }
        remainingSeats -= seats;
        System.out.println("Booking successful. Remaining seats: " +
remainingSeats);
    }

    public static void main(String[] args) {
        EventBooking booking = new EventBooking();

        try {
            booking.bookSeats(30); // invalid case
        } catch (SeatsUnavailableException e) {
            System.out.println(e.getMessage());
        }

        try {
            booking.bookSeats(10); // valid case
        } catch (SeatsUnavailableException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The seat check enforces a strict limit before confirming the booking.
- The exception message gives both requested and available seats, making the error user-friendly.
- By decrementing seats only after validation, I prevent negative inventory issues.

Future scope:

- Add waitlist support when events are full.
- Handle group bookings with atomic operations (all-or-nothing).
- Synchronize bookings in a concurrent environment (multiple users booking at once).
- Connect with payment gateway for transactional integrity (seat confirmation only after payment success).

Scenario 22: Restricting gym access for expired memberships

In a gym membership system, only users with valid subscriptions should be allowed to enter. Allowing expired members would break business rules and lead to revenue leakage.

Answer:

I'd store the subscription's expiry date and compare it with the current date at the time of access. If the current date is past the expiry date, I'd throw a MembershipExpiredException.

Code:

```
import java.time.LocalDate;

class MembershipExpiredException extends Exception {
    public MembershipExpiredException(String message) {
        super(message);
    }
}

class GymAccess {
    private LocalDate expiryDate = LocalDate.of(2025, 8, 1); // expiry date

    public void enterGym(LocalDate currentDate) throws
MembershipExpiredException {
        if (currentDate.isAfter(expiryDate)) {
            throw new MembershipExpiredException(
                "Access denied. Membership expired on " + expiryDate
            );
        }
        System.out.println("Welcome to the gym!");
    }

    public static void main(String[] args) {
        GymAccess access = new GymAccess();

        try {
            access.enterGym(LocalDate.of(2025, 8, 9)); // expired
        } catch (MembershipExpiredException e) {
            System.out.println(e.getMessage());
        }

        try {
            access.enterGym(LocalDate.of(2025, 7, 20)); // valid
        } catch (MembershipExpiredException e) {
            System.out.println(e.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- Using LocalDate makes it straightforward to compare expiry with current dates.
- The exception message clearly tells the user why access is denied and when the membership expired.
- This ensures business rule enforcement at the entry point, keeping unauthorized users out.

Future scope:

- Add a renewal feature to extend memberships.
- Integrate with turnstiles or access cards for automatic validation.
- Notify users before their membership expiry.
- Store multiple membership types (monthly, yearly, trial) with different expiry logic.

Scenario 23: Restricting file uploads that exceed allowed size

In a file upload service, large files can impact storage, bandwidth, and processing times. To maintain stability, a maximum file size must be enforced.

Answer:

I'd compare the incoming file's size against a configured limit. If it exceeds, I'd throw a FileTooLargeException with a clear message, ensuring only valid-sized files are processed.

Code:

```
class FileTooLargeException extends Exception {
    public FileTooLargeException(String message) {
        super(message);
    }
}

class FileUploader {
    private final long maxFileSizeKB = 2048; // 2 MB

    public void uploadFile(long fileSizeKB) throws FileTooLargeException {
        if (fileSizeKB > maxFileSizeKB) {
            throw new FileTooLargeException(
                "Upload failed. File size (" + fileSizeKB + "KB) exceeds " +
maxFileSizeKB + "KB limit."
            );
        }
        System.out.println("File uploaded successfully.");
    }

    public static void main(String[] args) {
        FileUploader uploader = new FileUploader();

        try {
            uploader.uploadFile(3072); // too large
        } catch (FileTooLargeException e) {
            System.out.println(e.getMessage());
        }

        try {
            uploader.uploadFile(1024); // valid size
        } catch (FileTooLargeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- A custom exception makes the rule explicit and ensures only acceptable files enter the system.
- The message includes both the file size and limit, giving immediate clarity to the user.
- This check helps safeguard storage and maintain service performance.

Future scope:

- Allow different file size limits based on user type (e.g., free vs premium users).
- Add support for rejecting files by type (e.g., block .exe uploads).
- Integrate with cloud storage providers where limits can vary.
- Provide front-end validation before upload to reduce server-side load.

Scenario 24: Blocking hotel bookings for past dates

In hotel systems, letting users book rooms for dates that have already passed leads to confusion and support overhead. The rule is simple: only today or future dates are valid (depending on policy).

Answer:

I'll compare the requested booking date with the current date. If it's before today, I'll throw a PastDateBookingException with a clear message. This keeps invalid reservations out at the earliest point.

Code:

```
import java.time.Clock;
import java.time.LocalDate;

class PastDateBookingException extends Exception {
    public PastDateBookingException(String message) {
        super(message);
    }
}

class HotelBooking {
    private final Clock clock;

    public HotelBooking(Clock clock) {
        this.clock = clock;
    }

    public void bookRoom(LocalDate bookingDate) throws
PastDateBookingException {
        LocalDate today = LocalDate.now(clock);
        if (bookingDate.isBefore(today)) {
            throw new PastDateBookingException("Cannot book for past date: " +
bookingDate);
        }
        System.out.println("Room booked for " + bookingDate);
    }

    public static void main(String[] args) {
        HotelBooking booking = new HotelBooking(Clock.systemDefaultZone());
        try {
            booking.bookRoom(LocalDate.of(2025, 8, 1));
        } catch (PastDateBookingException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- I injected Clock so the “current date” is testable and not hard-wired to the system clock—super useful for unit tests and time-zone sensitive systems.
- The rule triggers only when bookingDate is strictly before today; if the business also disallows same-day bookings, the check can be !bookingDate.isAfter(today).
- Throwing a domain exception keeps this business rule explicit and easy to handle upstream (UI/API).

Future scope:

- Add lead-time rules (e.g., booking must be at least 24 hours in advance).
- Validate check-in/check-out ranges (start < end).
- Surface localized messages and time-zone aware comparisons for multi-region hotels.

Scenario 25: Enforcing maximum course registrations per student

Universities cap the number of courses a student can take in a term. The system should stop registrations once the limit is reached to avoid load and scheduling issues.

Answer:

I'll track the student's current registrations and a configured max. When the student tries to add another course, if the new total would exceed the limit, I'll throw MaxCourseLimitException.

Code:

```
class MaxCourseLimitException extends Exception {
    public MaxCourseLimitException(String message) {
        super(message);
    }
}

class CourseRegistration {
    private int registeredCourses = 4;
    private final int maxAllowed = 5;

    public void addCourse() throws MaxCourseLimitException {
        if (registeredCourses >= maxAllowed) {
            throw new MaxCourseLimitException(
                "Cannot register for more than " + maxAllowed + " courses. " +
                "Currently registered: " + registeredCourses
            );
        }
        registeredCourses++;
        System.out.println("Course added. Total registered: " + registeredCourses);
    }

    public static void main(String[] args) {
        CourseRegistration registration = new CourseRegistration();
        try {
            registration.addCourse(); // 5th — allowed
            registration.addCourse(); // 6th — should fail
        } catch (MaxCourseLimitException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The guard clause checks the constraint before mutating state, so the count never exceeds the cap.
- The exception message includes both the max and current totals—this cuts down back-and-forth with users.
- Keeping maxAllowed as a field allows different policies per program/semester.

Future scope:

- Different limits by student type (undergrad, graduate, audit).
- Prerequisite and schedule-conflict checks before incrementing.
- Atomic registration across multiple courses (all-or-nothing) to avoid partial states.

Scenario 26: Preventing late exam submissions

In an online exam system, enforcing strict end times is essential for fairness. If students could submit after the exam ends, it would compromise integrity and lead to disputes.

Answer:

I'd store the exam's official end time. When a student submits, I'd compare the submission time with the end time. If the submission is later, I'd throw a LateSubmissionException.

Code:

```
import java.time.LocalTime;

class LateSubmissionException extends Exception {
    public LateSubmissionException(String message) {
        super(message);
    }
}

class OnlineExam {
    private LocalTime examEndTime = LocalTime.of(12, 0); // exam ends at noon

    public void submitExam(LocalTime submissionTime) throws
LateSubmissionException {
        if (submissionTime.isAfter(examEndTime)) {
            throw new LateSubmissionException(
                "Submission denied. Exam ended at " + examEndTime
            );
        }
        System.out.println("Exam submitted successfully at " + submissionTime);
    }

    public static void main(String[] args) {
        OnlineExam exam = new OnlineExam();

        try {
            exam.submitExam(LocalTime.of(12, 15)); // late submission
        } catch (LateSubmissionException e) {
            System.out.println(e.getMessage());
        }

        try {
            exam.submitExam(LocalTime.of(11, 45)); // valid submission
        } catch (LateSubmissionException e) {
            System.out.println(e.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- By using LocalTime, I avoid manual calculations and let the API handle comparisons.
- Throwing a custom exception makes the rule explicit—late submissions are not a technical error, but a business rule violation.
- Including the official exam end time in the message gives students clarity.

Future scope:

- Use LocalDateTime if exams span across days.
- Implement auto-submission when the timer runs out.
- Provide configurable grace periods (e.g., 1–2 minutes for network delays).
- Log late submission attempts for audit and fairness checks.

Scenario 27: Blocking orders when stock is depleted

In warehouse systems, once an item's stock is depleted, the system must reject new orders. If not enforced, customers would pay for items that can't be delivered, creating operational issues.

Answer:

I'd maintain a stock count for each item. If the requested quantity is greater than the available stock (including zero), I'd throw a StockDepletedException.

Code:

```
class StockDepletedException extends Exception {
    public StockDepletedException(String message) {
        super(message);
    }
}

class InventorySystem {
    private int stock = 0; // item is currently out of stock

    public void placeOrder(int quantity) throws StockDepletedException {
        if (quantity > stock) {
            throw new StockDepletedException(
                "Order failed. Only " + stock + " items left in stock."
            );
        }
        stock -= quantity;
        System.out.println("Order placed. Remaining stock: " + stock);
    }

    public static void main(String[] args) {
        InventorySystem system = new InventorySystem();

        try {
            system.placeOrder(1); // invalid request since stock = 0
        } catch (StockDepletedException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- I kept stock private to ensure it's only modified through controlled methods.

- Throwing StockDepletedException gives a descriptive error instead of a silent failure.
- This keeps customer expectations realistic—no order is confirmed if items aren’t available.

Future scope:

- Add automatic notifications to restock items.
- Implement waitlists or pre-orders when items are out of stock.
- Support concurrent order requests safely with synchronization.
- Maintain an audit trail of failed orders due to depletion.

Scenario 28: Enforcing credit limits in a payment gateway

In credit card systems, every transaction must be validated against the available credit limit. If payments are allowed beyond the limit, it would break financial rules and lead to disputes.

Answer:

I'd store the available credit and compare it against the requested transaction amount. If the amount exceeds the available credit, I'd throw a CreditLimitExceededException.

Code:

```
class CreditLimitExceededException extends Exception {
    public CreditLimitExceededException(String message) {
        super(message);
    }
}

class PaymentGateway {
    private double availableCredit = 5000.0; // initial credit limit

    public void processPayment(double amount) throws
CreditLimitExceededException {
        if (amount > availableCredit) {
            throw new CreditLimitExceededException(
                "Transaction denied. Available credit: $" + availableCredit + ",
Requested: $" + amount
            );
        }
        availableCredit -= amount;
        System.out.println("Payment processed. Remaining credit: $" +
availableCredit);
    }

    public static void main(String[] args) {
        PaymentGateway gateway = new PaymentGateway();

        try {
            gateway.processPayment(6000.0); // invalid transaction
        } catch (CreditLimitExceededException e) {
            System.out.println(e.getMessage());
        }

        try {
            gateway.processPayment(2000.0); // valid transaction
        } catch (CreditLimitExceededException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```

Reasoning to interviewer:

- By checking before deduction, I ensure the balance never goes negative.
- The exception message includes both available and requested amounts, which is important for user-facing systems.
- This mirrors real-world credit card transactions where banks enforce limits before authorizing.

Future scope:

- Support dynamic limits (gold, platinum cards with different thresholds).
- Add transaction history for audit purposes.
- Integrate with fraud detection systems before approval.
- Handle concurrency if multiple transactions happen simultaneously.

Scenario 29: Preventing players from joining full game rooms

In online games, each room has a maximum player capacity. Allowing extra players breaks gameplay balance and creates technical issues.

Answer:

I'd maintain a count of current players. Before allowing a new player to join, I'd compare it with the max allowed. If full, I'd throw a RoomFullException.

Code:

```
class RoomFullException extends Exception {
    public RoomFullException(String message) {
        super(message);
    }
}

class GameRoom {
    private int currentPlayers = 5;  // already full
    private final int maxPlayers = 5;

    public void joinRoom() throws RoomFullException {
        if (currentPlayers >= maxPlayers) {
            throw new RoomFullException(
                "Room is full. Max allowed: " + maxPlayers + ", Current: " +
currentPlayers
            );
        }
        currentPlayers++;
        System.out.println("Joined the room. Players now: " + currentPlayers);
    }

    public static void main(String[] args) {
        GameRoom room = new GameRoom();

        try {
            room.joinRoom(); // invalid attempt
        } catch (RoomFullException e) {
            System.out.println(e.getMessage());
        }

        // Example with a non-full room
        GameRoom newRoom = new GameRoom() {
            { currentPlayers = 3; } // override for demo
        };
        try {
            newRoom.joinRoom(); // valid join
        } catch (RoomFullException e) {
            System.out.println(e.getMessage());
```

```
    }
  }
}
```

Reasoning to interviewer:

- The guard clause prevents state corruption (no player added if full).
- Exception carries contextual info: current vs. max players, which is useful for debugging or UI prompts.
- This reflects real-world game matchmaking logic where capacity rules must be enforced.

Future scope:

- Add waitlists so extra players can auto-join when a spot opens.
- Implement concurrency handling so multiple players can’t join at the same instant beyond limit.
- Introduce room categories (ranked, casual) with different capacities.
- Notify players when a room they wanted becomes available.

Scenario 30: Locking bank accounts after multiple failed login attempts

In banking applications, login security is critical. Allowing unlimited failed attempts exposes the system to brute-force attacks. A lockout mechanism ensures the account gets protected after repeated failures.

Answer:

I'd keep track of failed login attempts. If they exceed the maximum allowed, I'd lock the account and throw an AccountLockedException. Until the account is reset or unlocked by admin/user action, all further attempts should fail immediately.

Code:

```
class AccountLockedException extends Exception {
    public AccountLockedException(String message) {
        super(message);
    }
}

class BankLogin {
    private int failedAttempts = 0;
    private final int maxFailedAttempts = 3;
    private boolean accountLocked = false;

    public void login(String password) throws AccountLockedException {
        if (accountLocked) {
            throw new AccountLockedException(
                "Account is locked due to multiple failed login attempts."
            );
        }

        if (!"securePass".equals(password)) {
            failedAttempts++;
            if (failedAttempts >= maxFailedAttempts) {
                accountLocked = true;
                throw new AccountLockedException(
                    "Account locked after " + failedAttempts + " failed attempts."
                );
            }
            System.out.println("Incorrect password. Attempts left: "
                               + (maxFailedAttempts - failedAttempts));
            return;
        }

        failedAttempts = 0; // reset on successful login
        System.out.println("Login successful.");
    }
```

```
    public static void main(String[] args) {
        BankLogin loginSystem = new BankLogin();
        try {
            loginSystem.login("wrong1");
            loginSystem.login("wrong2");
            loginSystem.login("wrong3"); // locks the account
            loginSystem.login("securePass"); // won't work, account locked
        } catch (AccountLockedException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- I used a simple counter for failed attempts and a boolean flag for account lock.
- The lock check happens before password validation to instantly block further attempts once locked.
- Resetting the counter on successful login prevents accidental lockouts due to minor typos.

Future scope:

- Add cooldown timers (unlock after X minutes).
- Log IP addresses for suspicious attempts.
- Integrate with 2FA or captcha after a few failed attempts.
- Allow admin dashboards to unlock accounts manually.

Scenario 31: Preventing double booking of seats in airline reservations

Airline systems must ensure each seat is uniquely assigned. Allowing multiple passengers to book the same seat creates chaos during check-in.

Answer:

I'd create a custom exception SeatAlreadyBookedException. If a passenger attempts to reserve a seat that's already taken, this exception would be thrown with a clear message.

Code:

```
class SeatAlreadyBookedException extends Exception {
    public SeatAlreadyBookedException(String msg) {
        super(msg);
    }
}

class FlightReservation {
    private boolean seatBooked = true; // seat already reserved

    public void reserveSeat() throws SeatAlreadyBookedException {
        if (seatBooked) {
            throw new SeatAlreadyBookedException(
                "Seat is already booked. Please select another."
            );
        }
        seatBooked = true;
        System.out.println("Seat reserved successfully.");
    }

    public static void main(String[] args) {
        FlightReservation reservation = new FlightReservation();
        try {
            reservation.reserveSeat(); // invalid attempt
        } catch (SeatAlreadyBookedException e) {
            System.out.println(e.getMessage());
        }

        // Example with a free seat
        FlightReservation freeSeat = new FlightReservation() {
            { seatBooked = false; } // mark as available
        };
        try {
            freeSeat.reserveSeat(); // valid attempt
        } catch (SeatAlreadyBookedException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```

Reasoning to interviewer:

- Each seat has a state (booked or not). The system checks before confirming reservation.
- Throwing a descriptive exception keeps the booking process transparent and easy to debug.
- In a real-world system, this would be enforced at the database layer too (unique constraints)

Future scope:

- Manage seat availability with a seat map instead of a single boolean.
- Implement concurrency control (multiple users may try booking the same seat simultaneously).
- Add cancellation logic to free up previously booked seats.
- Integrate with payment gateway so seat allocation only confirms after successful payment.

Scenario 32: Preventing payments below the minimum amount in a payment gateway

In payment gateways, a minimum payment rule ensures that processing costs and business requirements are met. Allowing very small transactions may lead to inefficiency or even losses.

Answer:

I'd define a custom exception MinimumPaymentException. Before processing, I'd check if the payment is below the configured minimum. If it is, the exception is raised, stopping the transaction early.

Code:

```
class MinimumPaymentException extends Exception {
    public MinimumPaymentException(String msg) {
        super(msg);
    }
}

class PaymentProcessor {
    private static final double MIN_PAYMENT = 50.0; // threshold in currency units

    public void processPayment(double amount) throws
MinimumPaymentException {
        if (amount < MIN_PAYMENT) {
            throw new MinimumPaymentException(
                "Minimum payment of " + MIN_PAYMENT + " is required. Entered: " +
amount
            );
        }
        System.out.println("Payment of " + amount + " processed successfully.");
    }

    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();

        try {
            processor.processPayment(25.0); // too low
        } catch (MinimumPaymentException e) {
            System.out.println(e.getMessage());
        }

        try {
            processor.processPayment(100.0); // valid
        } catch (MinimumPaymentException e) {
            System.out.println(e.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- The check happens upfront to avoid unnecessary processing.
- Including both the required and entered amounts in the message improves user feedback.
- By making this a custom exception, we distinguish between business validation and technical errors (like network failures).

Future scope:

- Different minimums for various payment methods (e.g., card vs. UPI vs. wallet).
- Add configurable thresholds from a database or config file.
- Integrate currency conversion if supporting international payments.
- Handle partial payment or EMI options.

Scenario 33: Preventing re-enrollment in already completed courses

On e-learning platforms, once a course is completed, re-enrolling is usually unnecessary. Blocking redundant enrollments prevents confusion and misuse of resources.

Answer:

I'd create a custom exception CourseAlreadyCompletedException. If the system marks a course as completed for a student, and they attempt to enroll again, this exception would be thrown with a clear explanation.

Code:

```
class CourseAlreadyCompletedException extends Exception {
    public CourseAlreadyCompletedException(String msg) {
        super(msg);
    }
}

class EnrollmentSystem {
    private boolean completed = true; // course already marked as completed

    public void enrollInCourse(String courseName) throws
CourseAlreadyCompletedException {
        if (completed) {
            throw new CourseAlreadyCompletedException(
                "You have already completed the course: " + courseName
            );
        }
        completed = true; // mark as completed after enrollment
        System.out.println("Enrolled in course: " + courseName);
    }

    public static void main(String[] args) {
        EnrollmentSystem system = new EnrollmentSystem();

        try {
            system.enrollInCourse("Java Programming"); // invalid attempt
        } catch (CourseAlreadyCompletedException e) {
            System.out.println(e.getMessage());
        }

        // Example with not completed
        EnrollmentSystem newSystem = new EnrollmentSystem() {
            { completed = false; } // mark as incomplete for testing
        };
        try {
            newSystem.enrollInCourse("Data Structures"); // valid attempt
```

```
        } catch (CourseAlreadyCompletedException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The completed flag simulates whether the student has finished the course.
- Throwing an exception here enforces a clear business rule: no duplicate enrollments for completed courses.
- The message is user-friendly, explicitly mentioning the course name.

Future scope:

- Allow re-enrollment for refresher purposes but with special conditions (e.g., after a year).
- Add progress tracking instead of a simple boolean (percent complete).
- Integrate with certificates—only completed courses should generate them.
- Support multiple courses with a mapping of course → completion status.

Scenario 34: Preventing item removals beyond stock in a warehouse

In warehouse systems, removing more items than available creates mismatched records and operational chaos. The system must enforce a hard stop on such requests.

Answer:

I'd define a custom exception OverstockRemovalException. Whenever someone tries to remove more items than are currently in stock, this exception should be raised with a descriptive message.

Code:

```
class OverstockRemovalException extends Exception {
    public OverstockRemovalException(String msg) {
        super(msg);
    }
}

class Inventory {
    private int stock = 15; // initial stock

    public void removeItems(int quantity) throws OverstockRemovalException {
        if (quantity > stock) {
            throw new OverstockRemovalException(
                "Cannot remove " + quantity + " items. Only " + stock + " available."
            );
        }
        stock -= quantity;
        System.out.println(quantity + " items removed. Remaining stock: " + stock);
    }

    public static void main(String[] args) {
        Inventory inventory = new Inventory();

        try {
            inventory.removeItems(20); // invalid attempt
        } catch (OverstockRemovalException e) {
            System.out.println(e.getMessage());
        }

        try {
            inventory.removeItems(5); // valid attempt
        } catch (OverstockRemovalException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The guard clause prevents stock from ever going negative.
- Exception messages include both requested and available counts, making the issue obvious to the user.
- Encapsulation ensures stock changes only happen through validated logic.

Future scope:

- Add audit logging for failed removal attempts.
- Implement concurrency safety to handle simultaneous removals.
- Integrate with automated reordering when stock drops below a threshold.
- Maintain multi-item stock maps instead of a single integer.

Scenario 35: Blocking underage drivers in a car rental system

Car rentals are legally regulated, and age restrictions are one of the most important rules. Allowing underage drivers would violate laws and expose companies to liability.

Answer:

I'd create a custom exception UnderageDriverException. During rental requests, if the driver's age is below the legal driving age (18 in this case), the system should throw this exception immediately.

Code:

```
class UnderageDriverException extends Exception {
    public UnderageDriverException(String msg) {
        super(msg);
    }
}

class CarRental {
    private static final int LEGAL_DRIVING_AGE = 18;

    public void rentCar(int age) throws UnderageDriverException {
        if (age < LEGAL_DRIVING_AGE) {
            throw new UnderageDriverException(
                "You must be at least " + LEGAL_DRIVING_AGE + " to rent a car."
            );
        }
        System.out.println("Car rental approved for driver aged " + age + ".");
    }

    public static void main(String[] args) {
        CarRental rental = new CarRental();

        try {
            rental.rentCar(16); // invalid
        } catch (UnderageDriverException e) {
            System.out.println(e.getMessage());
        }

        try {
            rental.rentCar(21); // valid
        } catch (UnderageDriverException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The age check enforces a simple but critical business rule.
- By throwing a custom exception, I clearly separate legal violations from technical issues.
- Including the required age in the message avoids confusion for users.

Future scope:

- Support different legal age rules for different countries.
- Add premium services that raise the minimum age (e.g., luxury cars require 25+).
- Store renter profiles with verification (driver’s license validation).
- Integrate insurance eligibility checks alongside age checks.

Scenario 36: Blocking region-restricted content in a streaming app

Streaming platforms often enforce licensing restrictions where certain titles are not available in every country. Allowing access without checks would violate contracts and lead to legal issues.

Answer:

I'd create a custom exception ContentNotAvailableException. If the user tries to play content that's unavailable in their current region, this exception would be thrown with a clear message.

Code:

```
class ContentNotAvailableException extends Exception {
    public ContentNotAvailableException(String msg) {
        super(msg);
    }
}

class StreamingService {
    private boolean availableInRegion = false; // simulate region restriction

    public void playContent(String title) throws ContentNotAvailableException {
        if (!availableInRegion) {
            throw new ContentNotAvailableException(
                "The content '" + title + "' is not available in your region."
            );
        }
        System.out.println("Playing: " + title);
    }

    public static void main(String[] args) {
        StreamingService service = new StreamingService();

        try {
            service.playContent("The Great Adventure"); // restricted
        } catch (ContentNotAvailableException e) {
            System.out.println(e.getMessage());
        }

        // Example of available content
        StreamingService allowedService = new StreamingService() {
            { availableInRegion = true; }
        };
        try {
            allowedService.playContent("World Cinema Classics"); // allowed
        } catch (ContentNotAvailableException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```

Reasoning to interviewer:

- The region restriction is enforced at the service layer before playback starts.
- Custom exception makes it explicit that the error is due to content licensing, not a technical playback issue.
- The message includes the title name, so users know exactly which content is blocked.

Future scope:

- Integrate with user's geolocation/IP to determine region dynamically.
- Provide alternative recommendations if a title is unavailable.
- Support VPN/legal compliance mechanisms for handling blocked content.
- Store region availability in a database per title instead of a simple flag.

Scenario 37: Preventing duplicate job applications in a recruitment portal

On job portals, candidates shouldn't be able to apply for the same job more than once. Duplicate applications waste recruiter time and create bad candidate experiences.

Answer:

I'd create a DuplicateApplicationException. If the system detects that the candidate has already applied for the same job, it throws this exception instead of processing another application.

Code:

```
class DuplicateApplicationException extends Exception {
    public DuplicateApplicationException(String msg) {
        super(msg);
    }
}

class JobPortal {
    private boolean applicationSubmitted = true; // simulate already applied

    public void applyForJob(String jobTitle) throws DuplicateApplicationException {
        if (applicationSubmitted) {
            throw new DuplicateApplicationException(
                "You have already applied for the job: " + jobTitle
            );
        }
        applicationSubmitted = true;
        System.out.println("Application submitted for: " + jobTitle);
    }

    public static void main(String[] args) {
        JobPortal portal = new JobPortal();

        try {
            portal.applyForJob("Software Engineer"); // duplicate attempt
        } catch (DuplicateApplicationException e) {
            System.out.println(e.getMessage());
        }

        // Example for a fresh application
        JobPortal newPortal = new JobPortal() {
            { applicationSubmitted = false; }
        };
        try {
            newPortal.applyForJob("Data Scientist"); // valid attempt
        } catch (DuplicateApplicationException e) {
```

```
            System.out.println(e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- A boolean flag is used here for simplicity, but in reality, it would be a database lookup to check if the user already applied.
- Throwing a custom exception keeps the validation rule explicit and separate from technical errors.
- The exception message mentions the job title to make the rejection clear.

Future scope:

- Store all applied job IDs per user and check against that list.
- Allow candidates to update or withdraw applications instead of blocking outright.
- Integrate with recruiters' dashboards to prevent duplicate resumes being sent.
- Notify users with status updates (e.g., "Application already in progress").

Scenario 38: Keep multi-threaded document processing running even if one document fails

You're processing many documents in parallel. One bad file (I/O error, parse error) shouldn't bring the whole run down. Also, if any thread forgets to catch an exception, you still want a global safety net to log it.

Answer:

I'll do two things:

1. Wrap each thread's work in a try–catch so a failure on one document is logged and doesn't kill sibling threads.
2. Register a global UncaughtExceptionHandler using Thread.setDefaultUncaughtExceptionHandler(...) so if any exception escapes a thread's run(), it's still logged instead of crashing the process.

This gives per-task resilience and a process-wide safety net.

Code:

```
import java.util.logging.Logger;

class DocumentProcessingException extends Exception {
    public DocumentProcessingException(String message) {
        super(message);
    }
}

public class DocumentProcessor {
    private static final Logger logger =
Logger.getLogger(DocumentProcessor.class.getName());

    public void processDocument(String docName) throws
DocumentProcessingException {
        if ("bad_doc.txt".equals(docName)) {
            throw new DocumentProcessingException("Failed to process: " +
docName);
        }
        System.out.println("Successfully processed: " + docName);
    }

    public static void main(String[] args) {
        // Global safety net for any exception that leaks out of a thread
        Thread.setDefaultUncaughtExceptionHandler((t, e) ->
            logger.severe("Unhandled exception in thread " + t.getName() + ": " + e));
```

```
        String[] documents = {"doc1.txt", "bad_doc.txt", "doc2.txt"};
        DocumentProcessor processor = new DocumentProcessor();

        for (String doc : documents) {
            Thread worker = new Thread(() -> {
                try {
                    processor.processDocument(doc);
                } catch (DocumentProcessingException e) {
                    // Local handling so siblings continue
                    logger.warning("Error processing '" + doc + "': " + e.getMessage());
                }
            }, "doc-worker-" + doc);
            worker.start();
        }
    }
}
```

Reasoning to interviewer:

- Per-thread try–catch localizes failures. One bad input doesn't poison the batch.
- setDefaultUncaughtExceptionHandler is the last line of defense—great for logging programming errors that slip through.
- I named threads (doc-worker-<doc>) so logs are traceable during incidents.

Future scope:

- Use an ExecutorService with a fixed pool and CompletionService to collect results/metrics.
- Add a bounded queue and backpressure if producers outrun consumers.
- Emit structured logs/metrics (success/failure counts, latency) and a final summary.
- For heavy parsing, consider timeouts and cancellation with Future or virtual threads (Project Loom).

Scenario 39: Retry payments up to 3 times, then notify the user if they still fail

Payment gateways are flaky at times—network hiccups, transient timeouts. We should retry a handful of times, with short pauses, then fail cleanly and inform the customer.

Answer:

I'll wrap the payment call in a small retry loop (max 3). On each failure, I'll log, sleep briefly, and try again. If all attempts fail, I'll throw PaymentTimeoutException. The caller will log the final failure and trigger a customer notification. In real systems, I'd also pass an idempotency key so retries don't double-charge.

Code:

```
class PaymentTimeoutException extends Exception {
    public PaymentTimeoutException(String message) {
        super(message);
    }
}

public class SubscriptionPaymentProcessor {
    private final int maxRetries = 3;

    public void attemptPayment() throws PaymentTimeoutException {
        int tries = 0;
        while (tries < maxRetries) {
            try {
                System.out.println("Processing payment attempt #" + (tries + 1) + "...");
                // Simulate transient failures for first two attempts
                if (tries < 2) {
                    throw new PaymentTimeoutException("Payment service timed out.");
                }
                System.out.println("Payment completed successfully!");
                return; // success
            } catch (PaymentTimeoutException e) {
                tries++;
                if (tries == maxRetries) {
                    // give up after the last attempt
                    throw e;
                }
                System.out.println("Retrying soon... (will attempt #" + (tries + 1) + ")");
                try {
                    Thread.sleep(800); // simple backoff; could be exponential
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    // If interrupted, propagate a meaningful failure
                    throw new PaymentTimeoutException("Retry interrupted");
                }
```

```
            }
        }
    }

    public static void main(String[] args) {
        SubscriptionPaymentProcessor processor = new
SubscriptionPaymentProcessor();
        try {
            processor.attemptPayment();
        } catch (PaymentTimeoutException e) {
            System.out.println("Payment failed after retries: " + e.getMessage());
            System.out.println("Triggering user notification and logging for follow-
up...");
        }
    }
}
```

Reasoning to interviewer:

- Retries handle transient faults without involving the user.
- A small sleep avoids hammering the gateway; exponential backoff would be even better.
- Throwing after the final try makes failure explicit so the caller can notify the user and mark the invoice as unpaid.
- I kept the retry loop inside the use-case boundary so it’s easy to control per operation.

Future scope:

- Add exponential backoff with jitter, e.g., 500ms, 1s, 2s ± random.
- Use idempotency keys and correlation IDs to avoid duplicate charges and ease tracing.
- Implement a circuit breaker to stop flooding a down gateway.
- Emit events/metrics (attempt counts, reasons) for observability; route final failures to a dead-letter queue for ops review.

Scenario 40: Blocking bids below the minimum requirement in a marketplace auction

In bidding platforms, enforcing a minimum bid helps maintain fair value for items. If users can bid below the threshold, the auction system loses credibility and may frustrate sellers.

Answer:
I'd define a custom LowBidException. The auction method would check the incoming bid against the defined minimum. If the bid is too low, the exception is thrown with a clear message to inform the user that their bid was rejected.

Code:

```
class LowBidException extends Exception {
    public LowBidException(String message) {
        super(message);
    }
}

public class MarketplaceAuction {
    private static final double MINIMUM_BID = 150.0;

    public void submitBid(double bid) throws LowBidException {
        if (bid < MINIMUM_BID) {
            throw new LowBidException("Your bid must be at least $" + MINIMUM_BID
+ ". You entered: $" + bid);
        }
        System.out.println("Bid accepted: $" + bid);
    }

    public static void main(String[] args) {
        MarketplaceAuction auction = new MarketplaceAuction();

        try {
            auction.submitBid(120.0); // invalid
        } catch (LowBidException e) {
            System.out.println("Bid rejected: " + e.getMessage());
        }

        try {
            auction.submitBid(200.0); // valid
        } catch (LowBidException e) {
            System.out.println("Bid rejected: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The validation ensures no invalid bids enter the system.
- The exception makes the business rule violation explicit, rather than mixing with generic errors.
- The rejection message includes both the required minimum and the entered bid, giving the bidder clarity.

Future scope:

- Add logic for minimum bid increments (e.g., must be at least $10 higher than the last bid).
- Store bids in a list and track highest bidder.
- Introduce concurrency handling if multiple users place bids simultaneously.
- Notify losing bidders when a higher bid is placed.

Scenario 41: Rejecting orders when requested quantity exceeds available stock

In order management systems, overselling stock leads to unfulfilled orders and unhappy customers. The system must validate inventory before confirming any order.

Answer:

I'd create a custom exception NoStockAvailableException. During order processing, if the requested quantity exceeds the current stock, the system throws this exception and informs the user immediately.

Code:

```
class NoStockAvailableException extends Exception {
    public NoStockAvailableException(String message) {
        super(message);
    }
}

public class InventoryManager {
    private int currentStock = 5;

    public void processOrder(int orderQty) throws NoStockAvailableException {
        if (orderQty > currentStock) {
            throw new NoStockAvailableException("Only " + currentStock + " items are
in stock. Requested: " + orderQty);
        }
        currentStock -= orderQty;
        System.out.println("Order confirmed. Remaining stock: " + currentStock);
    }

    public static void main(String[] args) {
        InventoryManager manager = new InventoryManager();

        try {
            manager.processOrder(8); // invalid case
        } catch (NoStockAvailableException ex) {
            System.out.println("Order failed: " + ex.getMessage());
        }

        try {
            manager.processOrder(3); // valid case
        } catch (NoStockAvailableException ex) {
            System.out.println("Order failed: " + ex.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- The validation ensures stock cannot drop below zero.
- Throwing a custom exception highlights this as a business-rule violation, not a technical failure.
- The message includes both available and requested quantities, helping the user understand why their order failed.

Future scope:

- Implement partial order fulfillment (deliver what's in stock).
- Connect with suppliers for automatic restocking triggers.
- Add concurrency safety to handle multiple simultaneous orders.
- Maintain audit logs of failed order attempts for business insights.

Scenario 42: Handling invalid signup inputs without stopping other registrations

In signup workflows, both username and password are mandatory. If either is blank, the system should reject that signup but continue with the rest. This prevents a single invalid record from halting the entire batch.

Answer:

I'll define a checked exception InvalidFormInputException for validation errors. Inside register(), if username or password is empty, I'll throw the exception. The main loop will catch it, log the issue, and move on to the next record.

Code:

```
class InvalidFormInputException extends Exception {
    public InvalidFormInputException(String message) {
        super(message);
    }
}

public class SignupHandler {
    public void register(String user, String pass) throws InvalidFormInputException {
        if (user == null || user.isEmpty() || pass == null || pass.isEmpty()) {
            throw new InvalidFormInputException("Username and password cannot be
empty.");
        }
        System.out.println("Registration completed for: " + user);
    }

    public static void main(String[] args) {
        SignupHandler handler = new SignupHandler();
        String[] usernames = {"alice", "", "charlie"};
        String[] passwords = {"pass123", "abc", ""};

        for (int i = 0; i < usernames.length; i++) {
            try {
                handler.register(usernames[i], passwords[i]);
            } catch (InvalidFormInputException e) {
                System.out.println("Signup error: " + e.getMessage());
            }
        }
    }
}
```

Reasoning to interviewer:

- By using a checked exception, we force callers to explicitly handle validation issues.
- The loop demonstrates resilience: one bad signup doesn’t block the rest.
- Clear error messages improve troubleshooting and user feedback.

Future scope:

- Enforce stronger rules (minimum password length, email format).
- Log failed attempts separately for audit.
- Integrate with UI validation to catch most issues before hitting backend.
- Add rate limiting to prevent bot signups.

Scenario 43: Retrying connections to a remote server before failing

Remote servers can be temporarily unreachable. Instead of failing immediately, retrying a few times increases success rate. But if retries are exhausted, the system should stop gracefully and inform the user.

Answer:

I'll implement retry logic in connectAndFetch(). Each attempt will try connecting, and if it fails, it'll throw and catch a RemoteServerDownException. After three failed attempts, the method rethrows the exception to signal failure.

Code:

```
class RemoteServerDownException extends Exception {
    public RemoteServerDownException(String message) {
        super(message);
    }
}

public class DataFetcher {
    private final int maxAttempts = 3;

    public void connectAndFetch() throws RemoteServerDownException {
        int tries = 0;
        while (tries < maxAttempts) {
            try {
                System.out.println("Trying to reach the server... (Attempt " + (tries + 1) +
")");
                // Simulating failure for first two attempts
                if (tries < 2) {
                    throw new RemoteServerDownException("Server is not responding.");
                }
                System.out.println("Data retrieved successfully.");
                return; // success, exit loop
            } catch (RemoteServerDownException e) {
                tries++;
                if (tries == maxAttempts) {
                    throw e; // no more retries
                }
                System.out.println("Retrying... Attempt #" + (tries + 1));
                try {
                    Thread.sleep(1000); // small pause before retry
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    throw new RemoteServerDownException("Retry interrupted.");
                }
            }
        }
    }
```

```
    public static void main(String[] args) {
        DataFetcher fetcher = new DataFetcher();
        try {
            fetcher.connectAndFetch();
        } catch (RemoteServerDownException e) {
            System.out.println("Process failed: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- Retry loops handle transient errors, improving resilience.
- A pause (Thread.sleep) avoids spamming the server with immediate retries.
- Throwing after the last attempt makes it clear that the system gave up gracefully.
- In production, retries would include logging and possibly exponential backoff.

Future scope:

- Implement exponential backoff with jitter to spread retry load.
- Make retry count configurable.
- Log retries with timestamps for monitoring server reliability.
- Integrate with a circuit breaker to avoid hammering servers that are consistently down.

Scenario 44: Handling missing profile details during updates

When updating user profiles, every required field must be validated. If one is missing, the system should reject the update and tell the user exactly which field is invalid, instead of giving a generic error.

Answer:

I'll define a custom exception InvalidProfileDetailsException that accepts a clear message about the missing field. The update() method checks each required field and throws the exception if validation fails.

Code:

```
class InvalidProfileDetailsException extends Exception {
    public InvalidProfileDetailsException(String message) {
        super(message);
    }
}

public class ProfileUpdater {
    public void update(String fullName, String emailAddr) throws
InvalidProfileDetailsException {
        if (fullName == null || fullName.isEmpty()) {
            throw new InvalidProfileDetailsException("Full name is required.");
        }
        if (emailAddr == null || emailAddr.isEmpty()) {
            throw new InvalidProfileDetailsException("Email address is required.");
        }
        System.out.println("Profile successfully updated.");
    }

    public static void main(String[] args) {
        ProfileUpdater updater = new ProfileUpdater();
        try {
            updater.update("", "test@example.com"); // missing full name
        } catch (InvalidProfileDetailsException e) {
            System.out.println("Update error: " + e.getMessage());
        }

        try {
            updater.update("Alice Johnson", ""); // missing email
        } catch (InvalidProfileDetailsException e) {
            System.out.println("Update error: " + e.getMessage());
        }

        try {
            updater.update("Alice Johnson", "alice@example.com"); // valid
        } catch (InvalidProfileDetailsException e) {
            System.out.println("Update error: " + e.getMessage());
```

```
    }
  }
}
```

Reasoning to interviewer:

- Each field check is independent, so you know exactly what’s wrong.
- By throwing different messages, users receive targeted feedback.
- This approach keeps validation logic centralized inside the update method.

Future scope:

- Validate additional fields like phone numbers, addresses, or passwords.
- Collect multiple validation errors in a single run, instead of failing fast.
- Localize error messages for international platforms.
- Integrate with front-end validation for smoother UX.

Scenario 45: Managing file uploads with size violations and reporting them

In file upload workflows, large files must be rejected. But one bad file shouldn't stop other uploads. Instead, violations should be logged and summarized at the end.

Answer:

I'll define a FileSizeLimitExceeded exception. The upload() method throws it if a file is too large. In processFiles(), I'll catch these exceptions, add them to a violation list, and print a final report after all files are processed.

Code:

```
import java.util.ArrayList;
import java.util.List;

class FileSizeLimitExceeded extends Exception {
    public FileSizeLimitExceeded(String message) {
        super(message);
    }
}

public class FileUploader {
    private static final long MAX_FILE_SIZE = 5000; // 5 KB

    public void upload(String fileName, long fileSize) throws FileSizeLimitExceeded
{
        if (fileSize > MAX_FILE_SIZE) {
            throw new FileSizeLimitExceeded(
                "File '" + fileName + "' exceeds the allowed limit of " + MAX_FILE_SIZE
+ " bytes."
            );
        }
        System.out.println("Uploaded successfully: " + fileName);
    }

    public void processFiles(List<String> fileNames, List<Long> fileSizes) {
        List<FileSizeLimitExceeded> violations = new ArrayList<>();

        for (int i = 0; i < fileNames.size(); i++) {
            try {
                upload(fileNames.get(i), fileSizes.get(i));
            } catch (FileSizeLimitExceeded ex) {
                violations.add(ex);
                System.out.println("Violation: " + ex.getMessage());
            }
        }
```

```
        if (!violations.isEmpty()) {
            System.out.println("\n--- File Size Violation Report ---");
            for (FileSizeLimitExceeded ex : violations) {
                System.out.println(ex.getMessage());
            }
        }
    }

    public static void main(String[] args) {
        List<String> files = List.of("invoice.pdf", "large_report.pdf", "notes.txt");
        List<Long> sizes = List.of(3000L, 7000L, 2500L); // large_report.pdf exceeds
limit

        FileUploader uploader = new FileUploader();
        uploader.processFiles(files, sizes);
    }
}
```

Reasoning to interviewer:

- This design ensures resilience: one bad file doesn't stop the batch.
- Violations are collected for reporting, instead of being lost in console spam.
- Users get both immediate feedback and a consolidated summary at the end.

Future scope:

- Export violation reports to a log file or monitoring system.
- Add support for file type restrictions (e.g., block .exe).
- Parallelize uploads using threads or executors while still aggregating violations.
- Show progress bars and counts of success vs. failures for better UX.

Scenario 46: Processing a batch while skipping corrupted records

In data pipelines, some records will be bad—wrong format, missing fields, or clearly corrupt. The pipeline shouldn't stop; it should flag the bad ones, log them, and continue with the rest.

Answer:

I'll introduce a checked CorruptedDataException and throw it whenever a record fails validation. In the batch loop, I'll catch this exception per record, log it, and keep going. That way, a few bad apples don't halt the whole batch.

Code:

```
import java.util.ArrayList;
import java.util.List;

class CorruptedDataException extends Exception {
    public CorruptedDataException(String message) {
        super(message);
    }
}

public class DataHandler {
    public void handleData(String record) throws CorruptedDataException {
        // Example validation: reject literal "corrupted" or empty/blank
        if (record == null || record.isBlank() || "corrupted".equalsIgnoreCase(record)) {
            throw new CorruptedDataException("Corrupted record: '" + record + "'");
        }
        System.out.println("Record processed: " + record);
    }

    public void processBatch(List<String> records) {
        List<String> failures = new ArrayList<>();
        int success = 0;

        for (String record : records) {
            try {
                handleData(record);
                success++;
            } catch (CorruptedDataException ex) {
                failures.add(ex.getMessage());
                System.out.println("Error: " + ex.getMessage());
            }
        }

        System.out.println("\n--- Batch Summary ---");
        System.out.println("Processed OK: " + success);
        System.out.println("Failed: " + failures.size());
        failures.forEach(System.out::println);
```

```
    }

    public static void main(String[] args) {
        List<String> dataset = List.of("entry1", "corrupted", "entry2", " ", "entry3");
        new DataHandler().processBatch(dataset);
    }
}
```

Reasoning to interviewer:

- A checked exception makes it explicit that validation can fail and must be handled.
- Per-record try/catch isolates failures; the pipeline advances even when one record is bad.
- A short summary (success/failure) is practical for ops and alerting.

Future scope:

- Replace System.out with SLF4J/Logback and structured logs (record IDs, reasons).
- Add a dead-letter queue (DLQ) to store failed records for reprocessing.
- Pluggable validators so different data feeds can reuse the pipeline.
- Parallelize with an executor, but still isolate and report per-record failures.

Scenario 47: Continue image filters even if one fails

In an image pipeline (resize → compress → watermark), a single flaky step—say, watermark service down—shouldn't prevent the rest from running. Best effort still improves the output.

Answer:

I'll define an ImageFilter interface with apply(image). Each filter implements it. The processor iterates through filters and wraps each apply in try/catch so failures are logged and the chain keeps going.

Code:

```
interface ImageFilter {
    void apply(String image) throws Exception;
}

class ResizeFilter implements ImageFilter {
    @Override
    public void apply(String image) {
        System.out.println("Resizing: " + image);
        // resize logic...
    }
}

class CompressFilter implements ImageFilter {
    @Override
    public void apply(String image) {
        System.out.println("Compressing: " + image);
        // compression logic...
    }
}

class WatermarkFilter implements ImageFilter {
    @Override
    public void apply(String image) throws Exception {
        // Simulate a failure
        throw new Exception("Watermark service unavailable");
    }
}

public class ImageProcessor {
    public static void main(String[] args) {
        String image = "photo.jpg";
        ImageFilter[] pipeline = {
            new ResizeFilter(),
            new CompressFilter(),
            new WatermarkFilter()
        };
```

```
        int failures = 0;
        for (ImageFilter filter : pipeline) {
            try {
                filter.apply(image);
            } catch (Exception e) {
                failures++;
                System.out.println("Filter failed: " + e.getMessage());
            }
        }
        System.out.println("Image processing complete for: " + image +
                           " (failures: " + failures + ")");
    }
}
```

Reasoning to interviewer:

- The interface cleanly models a pipeline; adding/removing filters is trivial.
- Per-filter try/catch isolates failures—others still run, which is crucial for large batches.
- The final tally is helpful for monitoring and conditional actions (e.g., re-run watermark later).

Future scope:

- Add a result object per image capturing which steps succeeded/failed.
- Introduce priorities or “mandatory vs. optional” filters (fail-fast only on mandatory).
- Use an ExecutorService for parallel batches; keep per-image isolation.
- Emit metrics (success rate by filter) and set up alerts when a filter’s failure rate spikes.

Scenario 48: Handling failed payments without stopping queued transactions

In a payment gateway, not all transactions succeed—some fail due to insufficient funds, expired cards, or network issues. The gateway must handle failures gracefully, log them, and keep processing the rest of the queue.

Answer:

I'll define a custom PaymentFailedException that is thrown for failed payments. The processor loops over all transactions, and each failure is caught and logged without interrupting other transactions.

Code:

```
class PaymentFailedException extends Exception {
    public PaymentFailedException(String message) {
        super(message);
    }
}

class PaymentService {
    public void processPayment(String transactionId) throws
PaymentFailedException {
        // Simulating a failure for TXN002
        if ("TXN002".equals(transactionId)) {
            throw new PaymentFailedException("Insufficient balance for " +
transactionId);
        }
        System.out.println("Payment successful: " + transactionId);
    }
}

public class PaymentProcessor {
    public static void main(String[] args) {
        String[] transactions = { "TXN001", "TXN002", "TXN003" };
        PaymentService service = new PaymentService();

        int failedCount = 0;
        for (String txn : transactions) {
            try {
                service.processPayment(txn);
            } catch (PaymentFailedException e) {
                failedCount++;
                System.out.println("Error: " + e.getMessage());
            }
        }

        System.out.println("\n--- Payment Summary ---");
        System.out.println("Total transactions: " + transactions.length);
```

```
        System.out.println("Failed: " + failedCount);
    }
}
```

Reasoning to interviewer:

- Each payment is validated in isolation, so one failure doesn’t poison the queue.
- Logging failures keeps transparency and allows retry/reconciliation later.
- The summary report helps track overall processing health.

Future scope:

- Retry transient failures (e.g., network errors) with backoff.
- Push failed transactions to a “retry queue” or dead-letter topic.
- Integrate with external monitoring/alerting for failures above a threshold.
- Add transaction states (success, failed, pending) for audit trails.

Scenario 49: Processing multiple book requests even if some are unavailable

In online libraries, not every book requested will be available. Instead of failing the entire batch, the system should deny the unavailable request and continue serving others.

Answer:

I'll create a BookNotAvailableException to signal unavailable books. Each request is handled in a loop, and failures are logged individually while continuing to process remaining requests.

Code:

```
class BookNotAvailableException extends Exception {
    public BookNotAvailableException(String title) {
        super("Book not available: " + title);
    }
}

class Library {
    public void issueBook(String title) throws BookNotAvailableException {
        if ("The Lost World".equalsIgnoreCase(title)) {
            throw new BookNotAvailableException(title);
        }
        System.out.println("Book issued: " + title);
    }
}

public class LibrarySystem {
    public static void main(String[] args) {
        String[] requests = { "Java Basics", "The Lost World", "OOP Principles" };
        Library library = new Library();

        int unavailable = 0;
        for (String title : requests) {
            try {
                library.issueBook(title);
            } catch (BookNotAvailableException e) {
                unavailable++;
                System.out.println("Error: " + e.getMessage());
            }
        }

        System.out.println("\n--- Request Summary ---");
        System.out.println("Total requests: " + requests.length);
        System.out.println("Unavailable: " + unavailable);
    }
}
```

Reasoning to interviewer:

- Exceptions are thrown for business-rule violations (book unavailable) rather than technical errors.
- Handling per-request ensures system resilience and good user experience.
- Final summary improves transparency, giving users and admins a clear picture.

Future scope:

- Add a waitlist feature so unavailable requests can be fulfilled later.
- Track stock counts per book, not just availability booleans.
- Integrate with notifications (email/SMS) when a book becomes available.
- Implement concurrency safety for simultaneous book requests.

Scenario 50: Scheduler resilience when individual tasks fail

In a scheduling system, one task failing shouldn't stop the others. Each task must run in isolation, with errors logged and flagged for reporting.

Answer:

I'll represent tasks with a Task interface. The scheduler executes them in sequence, wrapping each call in a try–catch. Failed tasks are logged and marked as failed, while other tasks proceed normally.

Code:

```
interface Task {
    void execute() throws Exception;
}

class EmailTask implements Task {
    @Override
    public void execute() {
        System.out.println("Sending email...");
    }
}

class BackupTask implements Task {
    @Override
    public void execute() throws Exception {
        throw new Exception("Backup server not reachable");
    }
}

public class Scheduler {
    public static void main(String[] args) {
        Task[] tasks = { new EmailTask(), new BackupTask() };

        int failedCount = 0;
        for (Task task : tasks) {
            try {
                task.execute();
            } catch (Exception e) {
                failedCount++;
                System.out.println("Task failed: " + e.getMessage());
            }
        }

        System.out.println("\n--- Task Summary ---");
        System.out.println("Total tasks: " + tasks.length);
        System.out.println("Failed: " + failedCount);
    }
}
```

Reasoning to interviewer:

- Wrapping each task in try–catch prevents one failure from cascading.
- Task abstraction (Task interface) makes it easy to plug in new types of tasks.
- A simple summary shows reliability metrics.

Future scope:

- Persist task status to a database for monitoring.
- Add retry policies for transient errors.
- Use thread pools for concurrent scheduling.
- Implement priorities and cron-like scheduling rules.

Scenario 51: Resilient sensor data collection with faulty sensors

In IoT systems, not all sensors report correctly. Faulty ones should be skipped, but the rest of the data must still be collected for accuracy.

Answer:

I'll define a custom exception SensorDataException. Each reading attempt is wrapped in try–catch. If one sensor fails, the exception is logged, but the loop continues to process the others.

Code:

```
class SensorDataException extends Exception {
    public SensorDataException(String message) {
        super(message);
    }
}

class SensorReader {
    public void read(String sensorId) throws SensorDataException {
        if ("SENSOR_B".equals(sensorId)) {
            throw new SensorDataException("Failed to read from " + sensorId);
        }
        System.out.println("Reading from " + sensorId + ": OK");
    }
}

public class SensorSystem {
    public static void main(String[] args) {
        String[] sensors = { "SENSOR_A", "SENSOR_B", "SENSOR_C" };
        SensorReader reader = new SensorReader();

        int errors = 0;
        for (String id : sensors) {
            try {
                reader.read(id);
            } catch (SensorDataException e) {
                errors++;
                System.out.println("Error: " + e.getMessage());
            }
        }

        System.out.println("\n--- Sensor Collection Summary ---");
        System.out.println("Total sensors: " + sensors.length);
        System.out.println("Failed readings: " + errors);
    }
}
```

Reasoning to interviewer:

- Each sensor is isolated so one failure doesn’t break the collection process.
- Exception makes invalid data explicit and traceable.
- Summary provides quick insights into overall sensor health.

Future scope:

- Retry failed sensors with backoff before marking them as permanently failed.
- Integrate with a monitoring dashboard for real-time fault alerts.
- Store failed readings in a dead-letter queue for later investigation.
- Support parallel sensor reads with concurrency controls.